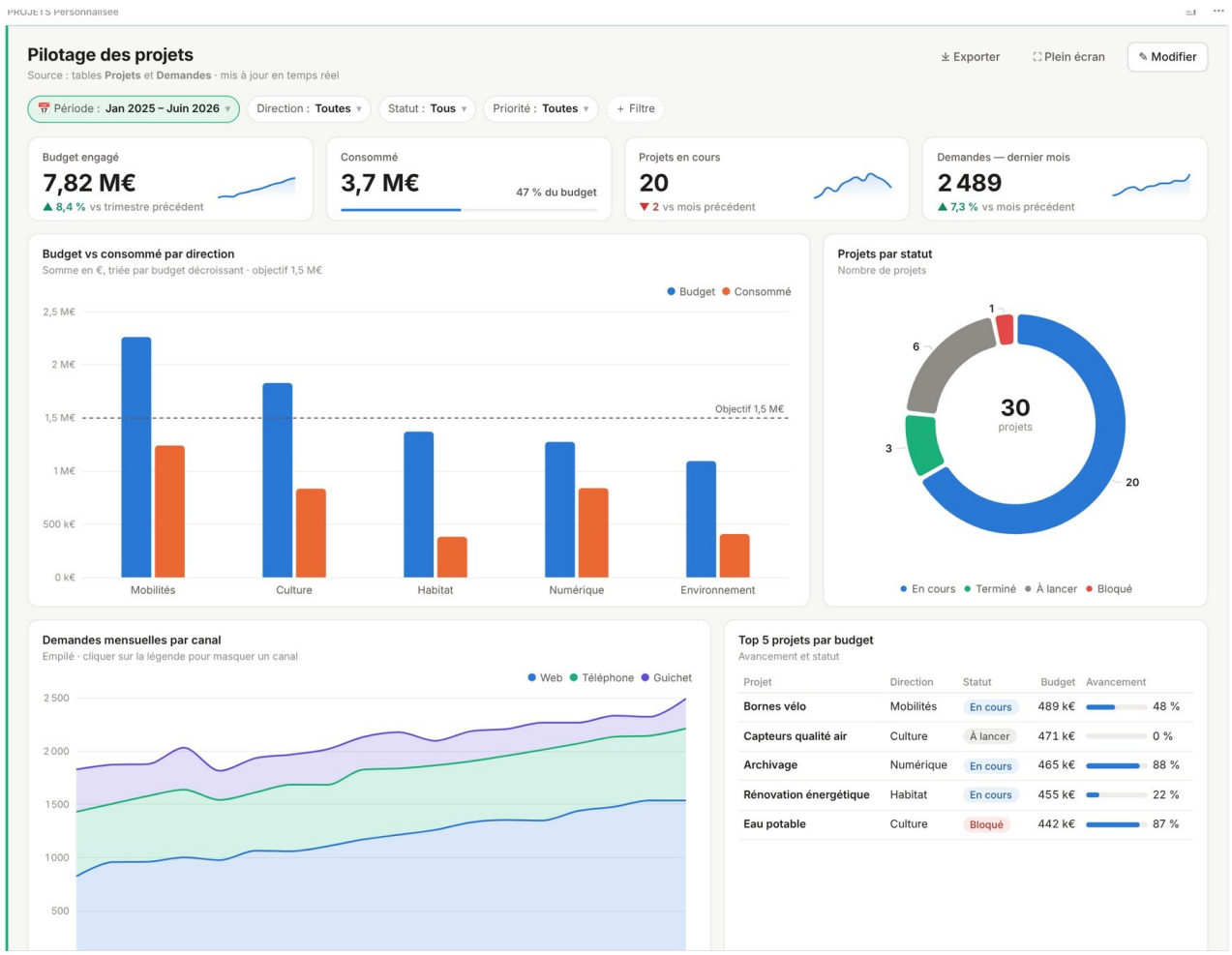
CAHIER DES CHARGES

Widget « Tableau de bord » pour Grist

Visualisations de données reposant sur Apache ECharts — benchmark, spécifications fonctionnelles, design et lotissement



Maquette fonctionnelle du widget, exécutée dans grist-core sur un document de démonstration (données réelles du document).

Version	Date	Statut	Périmètre
1.0	19 octobre 2026	Proposition à valider	Version initiale
1.1	20 octobre 2026	Proposition à valider	Ajouts : Notion vue Graphique / Dashboard (§ 3.1), Baserow et SeaTable (§ 3.4), IA (§ 3.8), compléments techniques (§ 7.2) — repérés « NOUVEAU v1.1 »

Sommaire

1. Contexte et objectifs....3
1.1 Contexte....3
1.2 Objectifs....3
1.3 Périmètre....3
1.4 Utilisateurs cibles....3

2. État des lieux : la visualisation dans Grist aujourd'hui....4
2.1 Le widget personnalisé « Advanced charts »....5

3. Benchmark....6
3.1 Notion — vue Graphique et vue Dashboard....6
3.2 Airtable — Interface Designer, mise en page « Dashboard »....8
3.3 NocoDB — Dashboards (bêta, 2025)....9
3.4 Baserow et SeaTable : alternatives open source européennes....10
3.5 Metabase — référence visuelle et de paramétrage....11
3.6 Grille comparative....12
3.7 Enseignements pour le design....12
3.8 Tendance 2026 : tableaux de bord générés par l'IA....12

4. Choix technologique : Apache ECharts....13
4.1 Comparatif des librairies....13
4.2 Preuve de faisabilité : le prototype « Rath »....13

5. Spécifications fonctionnelles....15
5.1 Principe général....15
5.2 Modes lecture et édition....15
5.3 Catalogue des tuiles....16
5.4 Paramétrage d'une tuile....18
5.5 Données et règles de calcul....19
5.6 Filtres globaux et interactions....19
5.7 Mise en page....19
5.8 Export et partage....19

6. Design et charte graphique....20
6.1 Principes....20
6.2 Palette....20
6.3 Jetons de design et anatomie d'une tuile....20
6.4 Mode sombre....21
6.5 Accessibilité....21

7. Exigences non fonctionnelles....22
7.1 Architecture technique....22
7.2 Compléments techniques vérifiés dans le code de grist-core....23

8. Lotissement, planning et risques....24
8.1 Critères d'acceptation....24

Annexe — Sources....25

1. Contexte et objectifs

1.1 Contexte

Grist est utilisé comme outil de gestion de bases de données relationnelles par des équipes métier. Les données y sont bien structurées (tables liées, formules Python, droits d'accès fins), mais leur **restitution visuelle** reste en retrait par rapport aux outils concurrents : les graphiques natifs reposent sur une intégration minimale de Plotly, sans notion de tableau de bord, d'indicateurs clés (KPI) ni de filtres globaux.

Les utilisateurs comparent spontanément Grist à Notion, Airtable ou NocoDB, qui proposent désormais des vues « Dashboard » soignées. L'objectif n'est pas de concurrencer un outil de BI complet, mais d'offrir **un rendu graphique au niveau d'un outil de gestion de données moderne** : lisible, élégant, configurable sans code.

1.2 Objectifs

1. Offrir un **widget personnalisé « Tableau de bord »** qui assemble, dans une même section Grist, des indicateurs, des graphiques, des tableaux et des textes organisés sur une grille.
2. S'appuyer sur **Apache ECharts** (licence Apache 2.0), la librairie utilisée par Metabase, pour un rendu de qualité professionnelle et un catalogue de graphiques étendu.
3. Proposer un **paramétrage visuel** inspiré de Metabase (onglets Données / Affichage / Axes) et validé par le prototype « Rath » déjà déployé.
4. Rester **natif Grist** : données lues en temps réel via l'API des widgets, respect des droits d'accès, configuration enregistrée dans le document.

1.3 Périmètre

Dans le périmètre	Hors périmètre
Widget personnalisé (iframe) installable sur grist-core et Grist SaaS Tuiles : KPI, graphiques ECharts, tableau, tableau croisé, texte, filtre Agrégations et regroupements temporels côté widget Filtres globaux, interactions (survol, zoom, clic → enregistrements) Mise en page en grille, mode lecture / édition, export PNG/SVG/CSV Thème clair / sombre, accessibilité, français et anglais	Requêtes SQL libres ou jointures arbitraires (on s'appuie sur les tables et colonnes de formule Grist) Alertes, envois programmés, abonnements e-mail Modification du cœur de Grist (le widget passe par l'API publique) Cartographie avancée (tuiles OSM, géocodage) — le widget Carte existant reste la référence Visualisations 3D, animations narratives

1.4 Utilisateurs cibles

Profil	Besoin principal	Attentes vis-à-vis du widget
Concepteur du document	Construire un tableau de bord sans code	Choix guidé du graphique, aperçu immédiat, valeurs par défaut pertinentes, glisser-déposer
Lecteur / décideur	Comprendre la situation en 30 secondes	KPI en tête, graphiques lisibles, filtres simples, export pour une réunion
Agent métier	Suivre son activité au quotidien	Filtrer par sa direction / période, cliquer sur une barre pour voir les lignes correspondantes

2. État des lieux : la visualisation dans Grist aujourd'hui

Pour évaluer l'existant dans des conditions réelles, un serveur **grist-core** a été compilé et lancé localement, puis un document « Pilotage des projets » a été créé (30 projets, 5 directions, 4 statuts, 54 relevés mensuels de demandes par canal). Trois graphiques natifs ont été ajoutés sur une page « Tableau de bord ».

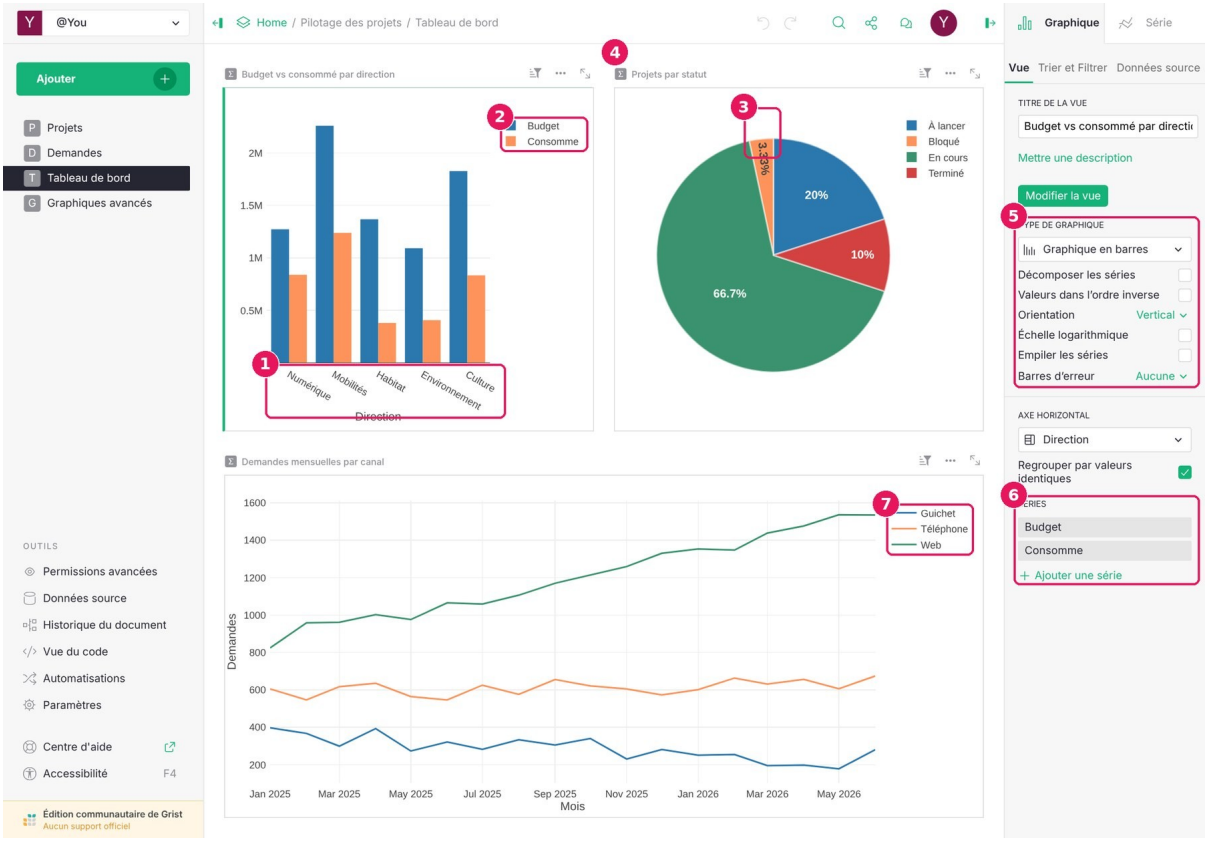


Figure 1 — *Graphiques natifs Grist (Plotly) sur le document de démonstration, panneau de configuration ouvert. Annotations : voir tableau ci-dessous.*

#	Observation
1	Libellés de l'axe X inclinés à 45° par défaut, aucun tri par valeur, pas de formatage monétaire des axes (« 2M », « 0.5M » à l'anglaise).
2	La légende affiche l'identifiant brut de la colonne (« Consomme » au lieu de « Consommé ») ; les couleurs sont celles de Plotly par défaut, non thématisables.
3	Camembert : étiquettes de pourcentage tronquées et pivotées dans les petites parts, pas de total central, pas de donut par défaut.
4	Aucun bandeau d'indicateurs (KPI), aucun filtre global : chaque graphique est une section indépendante, liée à une table récapitulative.
5	Options limitées à six cases (empiler, orientation, échelle log, barres d'erreur…) : pas de ligne d'objectif, d'étiquettes de valeur, de format de nombre, de palette.
6	Chaque série doit être une colonne d'une table récapitulative créée au préalable : l'agrégation (somme, moyenne, nombre) n'est pas choisie dans le graphique.
7	Lignes : rendu Plotly standard (pas d'aire, de lissage, d'infobulle formatée en français) ; l'ordre des champs détermine silencieusement le découpage des séries.

2.1 Le widget personnalisé « Advanced charts »

Grist propose aussi le widget « Advanced charts », qui embarque l'éditeur **react-chart-editor de Plotly**. Il est très puissant mais s'adresse à des profils techniques : interface en anglais, vocabulaire Plotly (traces, subplots, transforms), configuration trace par trace, aucune notion de tableau de bord.

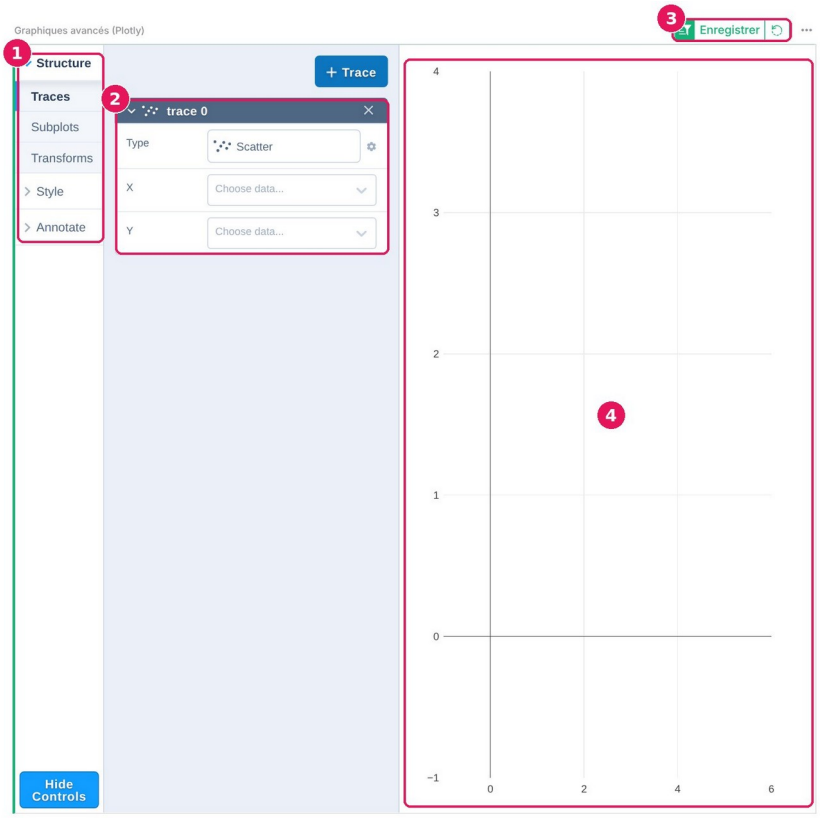


Figure 2 — *Widget « Advanced charts » : (1) arborescence Plotly en anglais, (2) configuration trace par trace, (3) enregistrement manuel, (4) zone de rendu vide tant que la trace n'est pas complète.*

Synthèse des manques

Pas de tableau de bord : ni grille, ni KPI, ni filtres globaux, ni texte de contexte entre les graphiques.

Rendu daté : palette, typographie et formats par défaut de Plotly ; pas de mode sombre cohérent ; pas de localisation des nombres.

Paramétrage contraint : dépendance aux tables récapitulatives, peu d'options d'affichage, éditeur avancé trop technique.

Catalogue restreint : 7 types natifs (barres, lignes, aires, nuage, secteurs, donut, Kaplan-Meier) ; pas de jauge, treemap, Sankey, cascade, heatmap, tableau croisé.

3. Benchmark

Le benchmark porte sur les trois outils de gestion de données cités (Notion, Airtable, NocoDB), sur deux alternatives open source européennes ajoutées en v1.1 (**Baserow**, **SeaTable**) et sur **Metabase**, retenu comme référence de qualité visuelle et de paramétrage (outil de BI open source, dont les graphiques reposent sur ECharts). Les captures proviennent des documentations publiques des éditeurs ; l'analyse UI se concentre sur ce qui est transposable dans Grist.

3.1 Notion — vue Graphique et vue Dashboard

Notion a introduit en 2024 une **vue Graphique** sur les bases de données (barres verticales et horizontales, lignes, donut, nombre), puis en mars 2026 une **vue Dashboard** qui assemble jusqu'à 12 widgets (4 par ligne) issus d'une ou plusieurs bases, avec des filtres globaux.

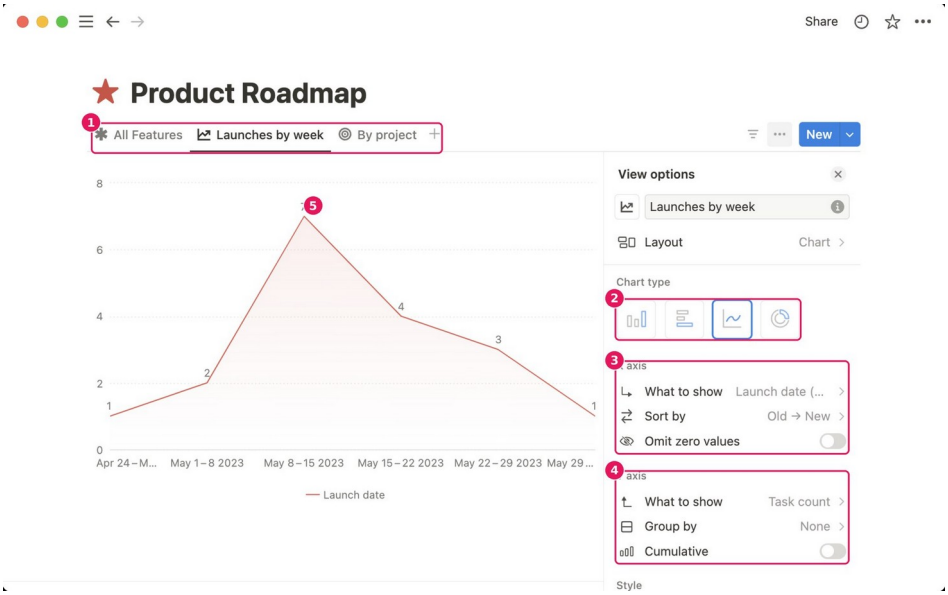


Figure 3 — *Notion — vue Graphique : (1) le graphique est une vue de la base, au même titre qu'un tableau ; (2) choix du type par icônes ; (3) axe X avec tri et masquage des zéros ; (4) axe Y avec regroupement et cumul ; (5) étiquettes de valeur sobres, aire en dégradé léger.*

Notion : vue Graphique ou vue Dashboard, deux niveaux distincts NOUVEAU v1.1

Dans Notion, une base de données s'affiche à travers des **vues** (tableau, Kanban, galerie, calendrier, frise, carte, formulaire…), présentées comme des onglets au-dessus des données. La **vue Graphique** (« Chart view », août 2024) est l'une de ces vues : elle représente **une base** sous la forme d'**un graphique**. La **vue Dashboard** (« Dashboard view », mars 2026) est d'une autre nature : c'est une vue **qui contient d'autres vues**. Chaque bloc, appelé widget, affiche une vue (graphique, tableau, Kanban, calendrier, frise) d'une base qui peut être différente d'un widget à l'autre, et l'ensemble partage des filtres globaux.

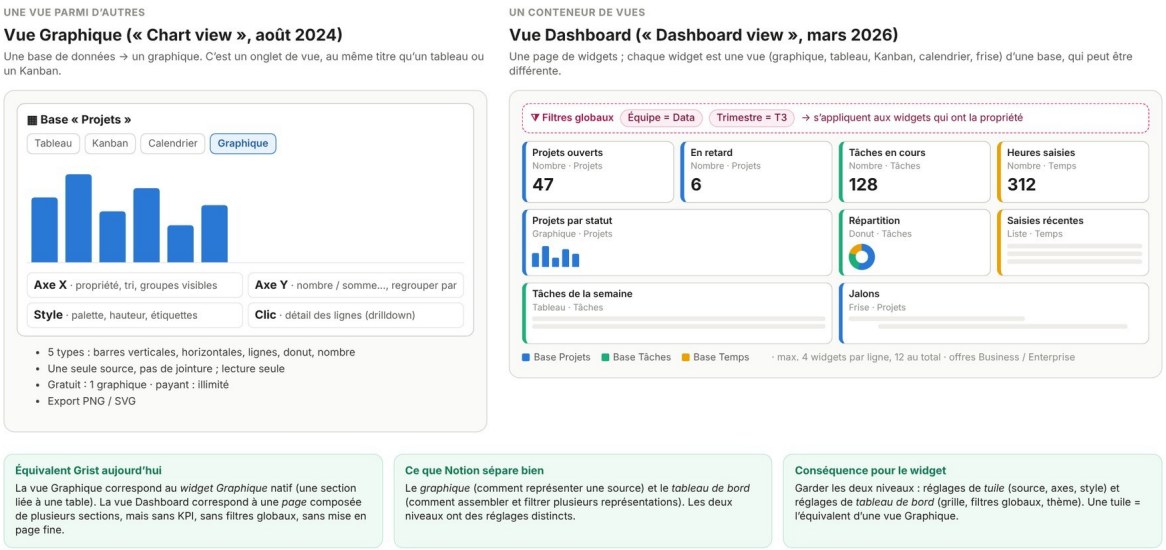


Figure 4 — *Schéma explicatif (réalisé pour ce document) : à gauche, la vue Graphique, une vue parmi les autres d'une base ; à droite, la vue Dashboard, un conteneur de widgets issus de plusieurs bases sous des filtres communs.*

Fonctionnement de la vue Graphique. On l'ajoute avec le « + » à côté des onglets de vue (choix « Graphique ») ou avec la commande `/chart` sur une page. Cinq types sont proposés : barres verticales, barres horizontales, lignes, donut et **nombre** (indicateur). Les réglages sont rangés par axe : axe X (propriété, tri, groupes visibles), axe Y (nombre de lignes ou somme, moyenne… d'une propriété, regroupement par une 2e propriété, cumul), puis style (palette, hauteur, quadrillage, noms d'axes, étiquettes de valeur, lissage, dégradé sous la courbe, valeur au centre du donut, légende). Depuis mars 2025, **un clic sur une barre ou une part ouvre le détail des lignes** sous forme de tableau, que l'on peut enregistrer comme nouvelle vue. On ne modifie pas les données depuis le graphique ; l'export se fait en PNG ou en SVG.

Fonctionnement de la vue Dashboard. On la crée comme n'importe quelle vue (« + », puis « Dashboard »), avec la commande `/dash`, ou en la faisant esquisser par Notion Agent à partir d'une description. Elle s'ouvre en **mode édition** : on ajoute un widget avec « + », en choisissant une vue existante ou en en créant une, puis on déplace les widgets, on ajuste les largeurs et la hauteur des rangées (4 widgets par rangée, 12 au maximum). En **mode lecture**, les filtres et tris appliqués restent locaux à l'utilisateur, sauf si un éditeur choisit « Enregistrer pour tous ». Un **filtre global** (« Filtrer plusieurs sources ») porte sur une propriété et ne s'applique qu'aux widgets dont la vue possède cette propriété. Les droits sont ceux des bases sous-jacentes.

Tableau 1 — *Notion — vue Graphique et vue Dashboard comparées (d'après le centre d'aide Notion, consulté le 20 octobre 2026)*

Critère	Vue Graphique	Vue Dashboard
Nature	Une vue d'une base, au même rang qu'un tableau	Une vue qui assemble d'autres vues (widgets)
Sources	Une seule base	Une ou plusieurs bases (une par widget)
Contenu	1 graphique : barres V/H, lignes, donut, nombre	Jusqu'à 12 widgets : graphiques, tableaux, Kanban, calendriers, frises
Réglages	Axes X/Y, regroupement, cumul, style	Mise en page, vue de chaque widget, filtres globaux
Filtres	Ceux de la vue	Globaux, multi-sources ; locaux au lecteur sauf « Enregistrer pour tous »
Interaction	Clic → détail des lignes ; lecture seule	Ouverture des pages, édition selon les droits

Critère	Vue Graphique	Vue Dashboard
Limites	200 groupes, 50 sous-groupes ; pas d'agrégats de relations (rollups), de boutons ni de fichiers sur les axes	12 widgets, 4 par rangée ; lenteur si les widgets chargent trop de lignes
Offre	Gratuit : 1 graphique ; offres payantes : illimité	Offres Business et Enterprise uniquement

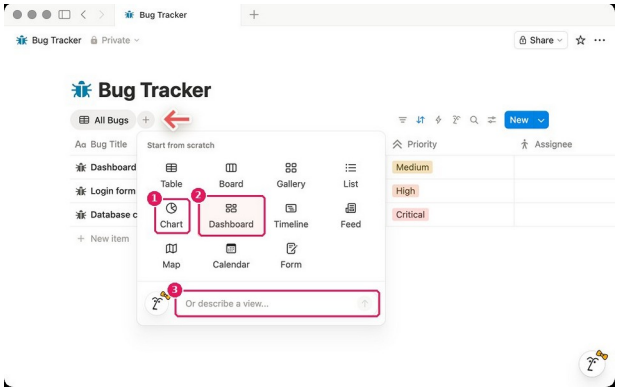


Figure 5 — *Menu « + » d'une base : (1) Graphique et (2) Dashboard sont deux types de vue distincts ; (3) une vue peut être décrite en langage naturel.*

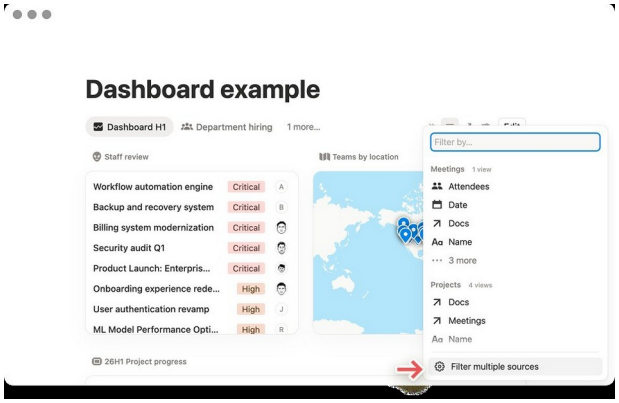


Figure 6 — *Vue Dashboard : widgets de types différents et filtre « Filtrer plusieurs sources » (flèche).*

Ce que cela change pour le widget Grist

Garder les deux niveaux : une **tuile** est l'équivalent d'une vue Graphique (une source, des axes, un style) ; le **tableau de bord** assemble les tuiles et porte la mise en page, les filtres globaux et le thème.

Filtre global = correspondance par table : comme dans Notion, un filtre ne s'applique qu'aux tuiles dont la table a la colonne visée. Le panneau du filtre indique, table par table, la colonne utilisée et les tuiles concernées.

Filtres du lecteur non enregistrés : ce que choisit un lecteur reste dans sa session ; seul un éditeur fixe les valeurs par défaut pour tous.

Pas de plafond strict de 12 tuiles, mais un avertissement au-delà de 16 tuiles et un chargement différé des tuiles hors écran.

À retenir de Notion

Peu d'options mais **excellentes valeurs par défaut** : couleurs douces, typographie soignée, grilles discrètes, aucun réglage nécessaire pour un rendu propre.

Panneau latéral court, organisé par axe ; le graphique se met à jour instantanément.

3.2 Airtable — Interface Designer, mise en page « Dashboard »

Airtable propose dans son Interface Designer une mise en page **Dashboard** fondée sur des éléments : nombres (KPI), graphiques (barres, lignes, secteurs, donut, nuage), filtres, listes et commentaires. Les KPI peuvent ouvrir les enregistrements sous-jacents.

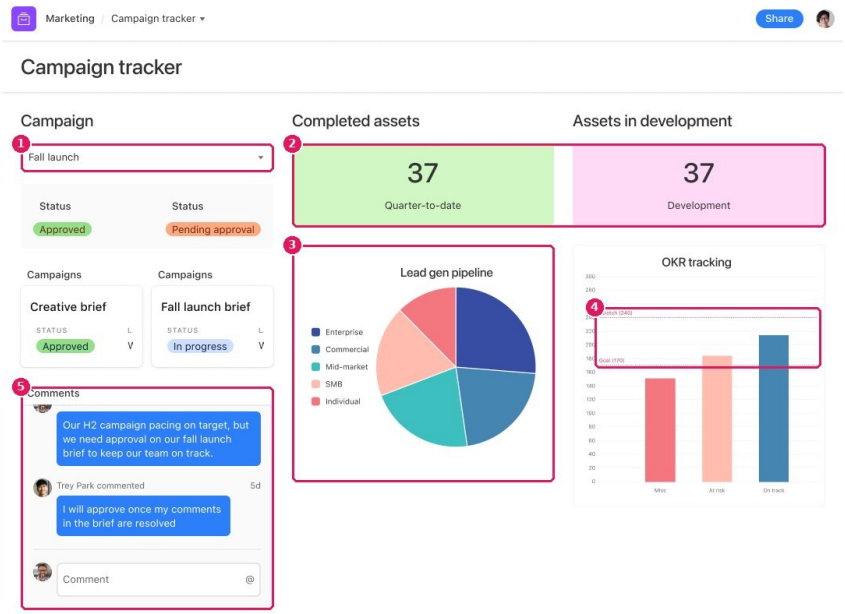


Figure 7 — *Airtable — tableau de bord « Campaign tracker » : (1) filtre déroulant qui pilote toute la page ; (2) KPI en grands chiffres sur fond coloré ; (3) secteurs avec légende latérale ; (4) lignes d'objectif « Goal » et « Stretch » ; (5) commentaires intégrés au tableau de bord.*

> **À retenir d'Airtable**
>
> Le **KPI est un objet de premier rang**, placé en haut et mis en valeur.
>
> Les **lignes d'objectif** et le **filtre de page** sont des fonctions simples à forte valeur pour les décideurs.

3.3 NocoDB — Dashboards (bêta, 2025)

NocoDB a lancé ses **Dashboards** en juillet 2025 (version 2025.07.0, en bêta sur les offres Plus et Business) : widgets Nombre, Barres, Lignes, Secteurs, Donut, Texte et iFrame, disposés par glisser-déposer, partageables par lien public ou protégé par mot de passe.

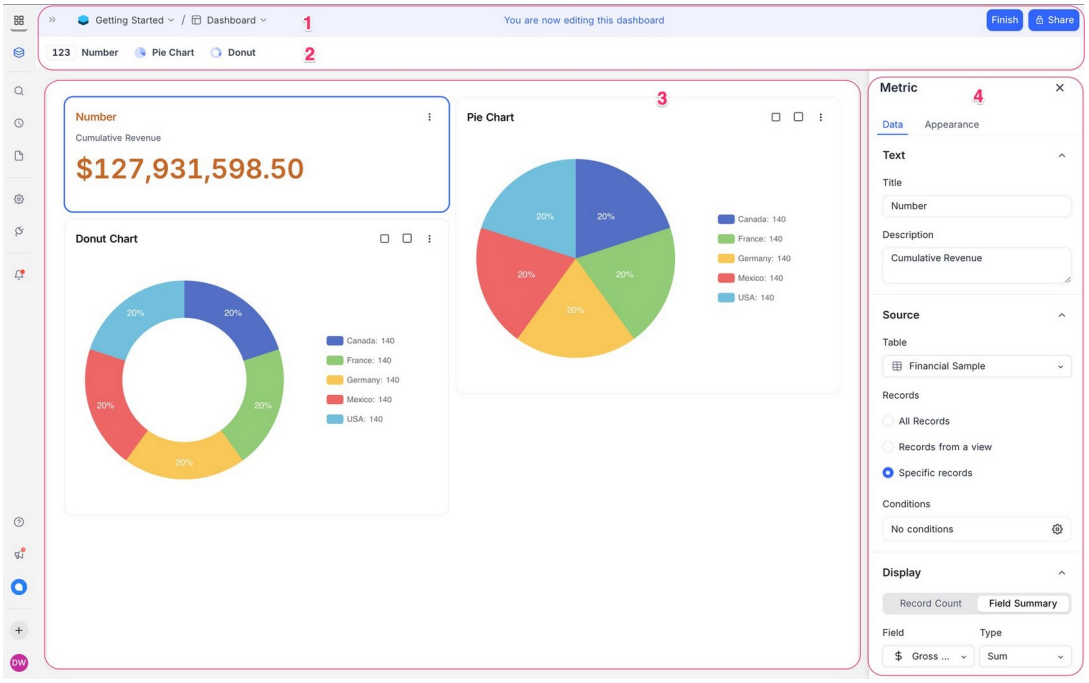


Figure 8 — *NocoDB — mode édition (annotations de l'éditeur) : (1) bandeau « You are now editing this dashboard », (2) barre d'ajout de widgets, (3) canevas en grille, (4) panneau de configuration Données / Apparence (titre, source, filtre d'enregistrements, agrégation).*

À retenir de NocoDB

Le modèle **« bandeau d'édition + barre d'ajout + canevas + panneau latéral »** est simple et immédiatement compréhensible.

Point faible : rendu encore basique (palette vive, peu d'options d'axes), ce que l'usage d'ECharts permet de dépasser.

3.4 Baserow et SeaTable : alternatives open source européennes NOUVEAU v1.1

Ces deux outils sont souvent comparés à Grist par les organisations publiques européennes qui recherchent une solution auto-hébergeable. Ils ont été ajoutés au benchmark.

Baserow (Pays-Bas, open source) a ouvert ses **Dashboards** en version 1.32 (2025, en bêta) avec deux widgets : **résumé** (un chiffre clé : total, moyenne, nombre) et **graphique en barres**. Les lignes ont suivi, puis les **secteurs et donuts** en version 1.35. La version 2.4 (30 septembre 2026) introduit une **grille** dans laquelle on déplace et redimensionne les widgets. Chaque widget a un titre, une description, une source (table ou vue), des séries agrégées (3 au maximum pour les barres et les lignes), un regroupement, un tri et un jeu de couleurs ; il se met à jour automatiquement et suit les droits des tables.

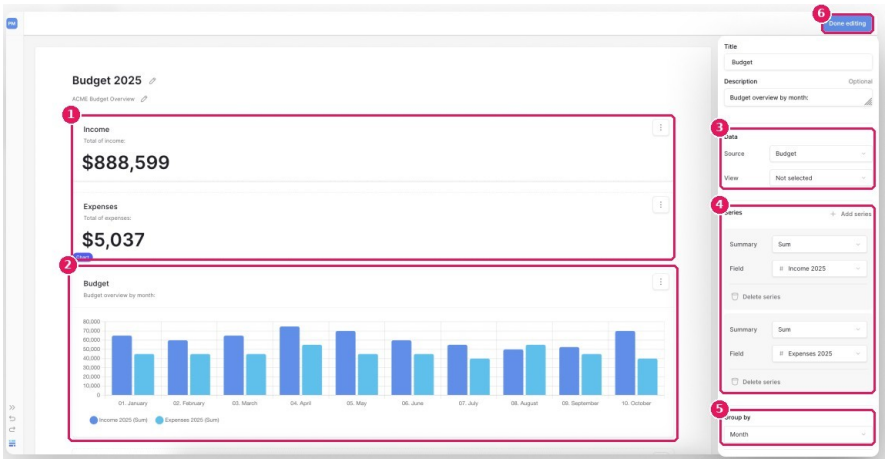


Figure 9 — *Baserow — édition d'un tableau de bord : (1) widgets « résumé » (KPI), (2) graphique en barres sélectionné, (3) titre et description, (4) source, vue et séries avec agrégation (somme), (5) regroupement par mois, (6) sortie du mode édition.*

SeaTable (Allemagne) propose un **module Statistiques** plutôt qu'un tableau de bord libre : barres, colonnes, lignes, secteurs, cartes, cartes de chaleur, compteur (jauge) et tableaux croisés, tous regroupés sur la page du module. Chaque statistique lit **une table à travers une vue** ; le module n'a pas de filtre propre, il faut donc préparer une vue filtrée. Les statistiques se recalculent à chaque modification des données.

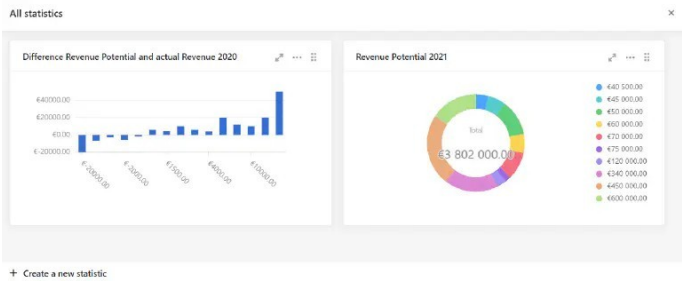


Figure 10 — *SeaTable — page du module Statistiques : graphiques enregistrés côte à côte.*

À retenir de Baserow et SeaTable

Baserow confirme le **socle attendu en 2026** : KPI, barres, lignes, secteurs/donut, grille déplaçable, panneau de réglages latéral. Le widget visé le dépasse (comparaison de périodes, filtres globaux, 20 types, export, interactions).

SeaTable montre qu'un **catalogue large** ne suffit pas : sans mise en page libre ni filtres globaux, le module reste

> une galerie de graphiques plutôt qu'un tableau de bord.

3.5 Metabase — référence visuelle et de paramétrage

Metabase n'est pas un outil de gestion de données mais un outil de BI open source. Il est retenu comme **référence de qualité** : ses graphiques sont rendus avec Apache ECharts (dépendance `echarts` 6.1 dans le dépôt officiel), son panneau de réglages est éprouvé par des millions d'utilisateurs et ses tableaux de bord combinent onglets, filtres, sections de texte et KPI comparatifs.

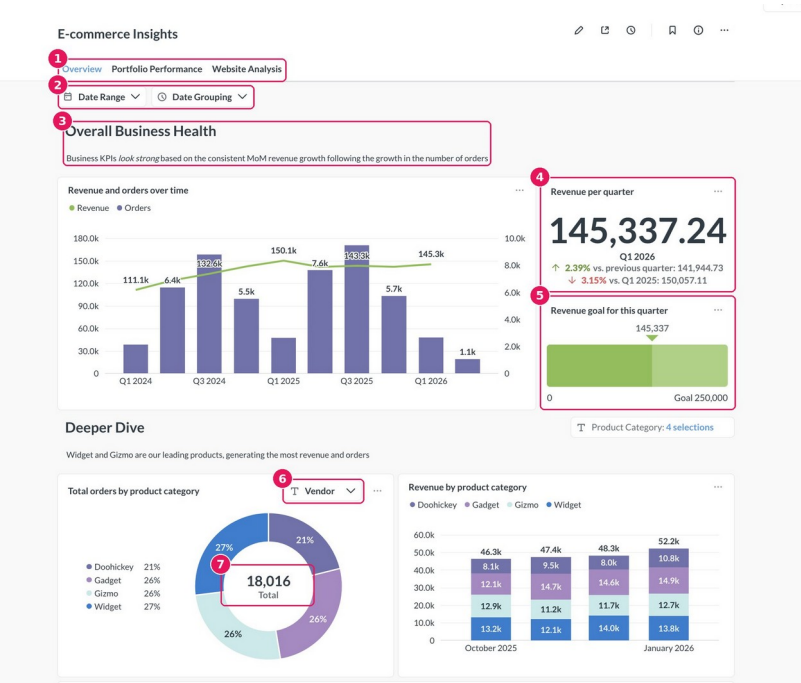


Figure 11 — *Metabase — tableau de bord : (1) onglets, (2) filtres globaux de période et de granularité, (3) titre de section et texte d'analyse, (4) KPI avec comparaison à la période précédente, (5) progression vers un objectif, (6) filtre propre à une tuile, (7) donut avec total central.*



Figure 12 — *Metabase — échantillon du catalogue de visualisations, toutes rendues avec ECharts.*

3.6 Grille comparative

Légende : ● complet · ◖ partiel · ○ absent · n.d. non documenté publiquement. La dernière colonne indique la cible du présent cahier des charges. En v1.1, la colonne Notion a été corrigée (graphique « Nombre », clic vers le détail des lignes, export PNG/SVG).

Tableau 2 — *Comparaison fonctionnelle des solutions étudiées (mise à jour v1.1)*

Critère	Grist actuel	Notion	Airtable	NocoDB	Baserow	SeaTable	Metabase	Cible Grist
Tableau de bord multi-tuiles	○	●	●	●	●	◖	●	●
KPI / grand nombre	○	●	●	●	●	◖	●	●
Comparaison à la période précédente	○	○	○	○	n.d.	n.d.	●	●
Filtres globaux	○	●	●	◖	n.d.	○	●	●
Nombre de types de graphiques	7	5	5	5	5	8	20+	20
Agrégation choisie dans le graphique	○	●	●	●	●	●	●	●
Ligne d’objectif, étiquettes de valeur	○	◖	●	○	n.d.	n.d.	●	●
Palette et thème personnalisables	○	◖	◖	◖	◖	n.d.	●	●
Mode sombre cohérent	◖	●	○	◖	n.d.	n.d.	●	●
Clic → enregistrements sous-jacents	◖	●	●	○	n.d.	n.d.	●	●
Export image / données	○	◖	○	○	n.d.	n.d.	●	●
Open source / auto-hébergeable	●	○	○	●	●	◖	●	●

3.7 Enseignements pour le design

- **Hiérarchie fixe** : titre et filtres → rangée de KPI → graphiques principaux → détail (tableau). Tous les outils convergent vers ce schéma.
- **Valeurs par défaut soignées plutôt qu’options nombreuses** (Notion) ; les options avancées restent accessibles mais repliées (Metabase).
- **Édition en contexte** : bandeau d’édition, tuile sélectionnée surlignée, panneau latéral à onglets (NocoDB, Metabase).
- **Sobriété graphique** : grilles très claires, pas de bordures d’axes, barres à coins arrondis, légende en pastilles, infobulles formatées.
- **Lien avec les données** : un clic sur un élément doit mener aux lignes concernées — c’est ce qui différencie un outil de gestion de données d’un simple générateur d’images.
- NOUVEAU **Deux niveaux de réglages** (Notion) : la tuile (source, axes, style) et le tableau de bord (mise en page, filtres globaux, thème).

3.8 Tendance 2026 : tableaux de bord générés par l’IA NOUVEAU v1.1

Les concurrents ajoutent la **création assistée** : Notion Agent peut produire une première version d’une vue Dashboard à partir d’une description, et **Omni**, l’assistant d’Airtable, génère des éléments de tableau de bord après avoir proposé un plan à valider (disponible sur toutes les offres, sans consommer de crédits IA). Le résultat reste modifiable dans l’éditeur habituel.

Recommandation : rendre le format de configuration du widget (§ 7.1) **public, documenté et validé par un schéma JSON**. Cela permettra, en lot V2, de proposer « Décrire le tableau de bord souhaité » : l'assistant IA de Grist (ou un autre modèle) produirait une configuration que l'utilisateur relit et ajuste dans le mode édition, sans aucun code généré.

4. Choix technologique : Apache ECharts

4.1 Comparatif des librairies

Tableau 3 — *Comparaison des principales librairies JavaScript de visualisation*

Critère	ECharts 6	Plotly.js (actuel)	Chart.js 4	Vega-Lite 5	Highcharts
Licence	Apache 2.0	MIT	MIT	BSD-3	Commerciale
Types de graphiques	40+ (dont Sankey, treemap, sunburst, jauge, carte, calendrier)	40+ (orienté scientifique)	8 de base + extensions	Grammaire, très large	40+
Qualité du rendu par défaut	Très bonne, moderne	Datée	Bonne	Sobre	Très bonne
Performances	Canvas ou SVG, échantillonnage, 100 k+ points	Bonne (WebGL)	Canvas, moyenne au-delà de 10 k	Moyenne	Bonne
Thèmes et mode sombre	Thèmes JSON, dark natif	Templates	Manuel	Config	Thèmes
Accessibilité	ARIA + motifs (decal)	Partielle	Partielle	Partielle	Module dédié
Localisation FR	Native (fr)	Locale à charger	Via Intl	Via format	Native
Taille (min+gz)	≈ 350 Ko, ou moins par import modulaire	≈ 1 Mo (basic ≈ 300 Ko)	≈ 70 Ko	≈ 400 Ko	≈ 120 Ko+
Références	Metabase, Apache Superset, Baidu	Grist, Dash	Nombreux SaaS	Observable, Jupyter	Grandes entreprises

Recommandation : Apache ECharts 6. C'est la seule option qui combine licence permissive compatible avec grist-core (Apache 2.0), catalogue complet (y compris jauges, Sankey, treemap, calendrier), rendu moderne par défaut, accessibilité intégrée (descriptions ARIA, motifs « decal » pour le daltonisme) et une référence directe dans le même segment (Metabase). Le rendu **SVG** sera utilisé par défaut (net à l'impression, exportable), le rendu Canvas au-delà de 5 000 points.

Alternatives écartées : Chart.js (catalogue trop limité), Highcharts (licence commerciale incompatible avec la distribution open source), Vega-Lite (excellente grammaire mais paramétrage peu adapté à une interface sans code), Plotly (déjà présent, mais c'est précisément son rendu qui est jugé insuffisant).

4.2 Preuve de faisabilité : le prototype « Rath »

Un prototype autonome (dépôt `Rath`, déployé sur Vercel) reproduit le **module de création de visualisation de Metabase** sur ECharts 6, React 18 et Mantine : import CSV, étape « Résumer » (agrégation, regroupement, éclatement), sélecteur de 20 types et panneau de réglages à trois onglets. Il démontre que la qualité visuelle de Metabase est atteignable en quelques milliers de lignes de code, sans dépendance à Metabase.



Figure 13 — *Prototype Rath — galerie des graphiques rendus par ECharts (captures de l'application déployée sur rath-rath-client-8vrf.vercel.app).*

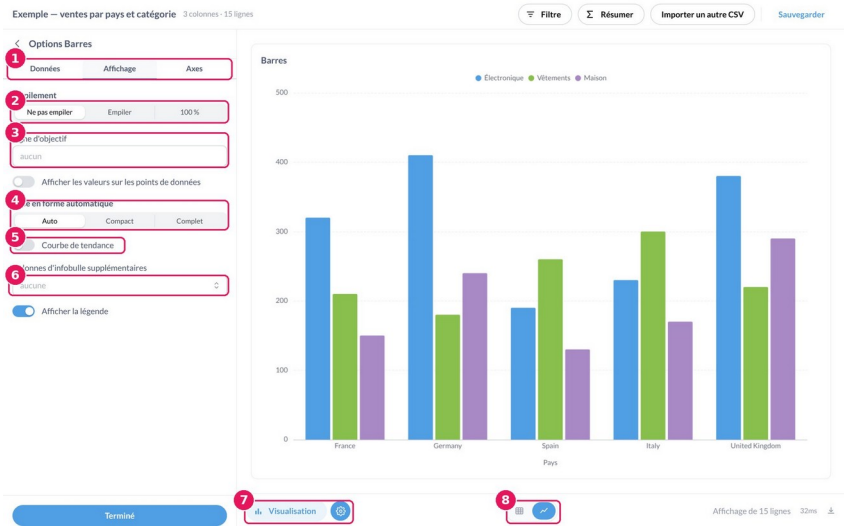


Figure 14 — *Prototype Rath — panneau « Affichage » : (1) onglets Données / Affichage / Axes, (2) empilement, (3) ligne d'objectif, (4) mise en forme automatique des nombres, (5) courbe de tendance, (6) colonnes d'infobulle, (7) bascule type de graphique / réglages, (8) bascule tableau / graphique.*

Ce que le widget Grist réutilise du prototype

`viz/frame.ts` : pipeline d'agrégation, de tri et d'éclatement (breakout) ; `viz/options/*` : constructeurs d'options ECharts par famille (cartésien, secteurs, treemap, Sankey, jauge…).

`viz/settings.ts` et `SettingsPanel.tsx` : modèle des réglages et panneau à onglets ; `viz/auto-display.ts` : recommandation du type de graphique selon les colonnes.

Adaptations nécessaires : source de données Grist au lieu du CSV, types de colonnes Grist (Référence, Choix, Liste de choix, Date, Numérique avec format), grille de tuiles, filtres globaux.

5. Spécifications fonctionnelles

5.1 Principe général

Le tableau de bord est **un widget personnalisé Grist** occupant une section (en pratique, une page entière). Il contient une **grille de tuiles** ; chaque tuile lit une table du document, applique ses propres agrégations et filtres, puis se rend avec ECharts. La configuration complète est enregistrée dans les options du widget (`grist.setOptions`), donc **versionnée avec le document**, copiée avec lui et soumise à l'historique d'annulation.

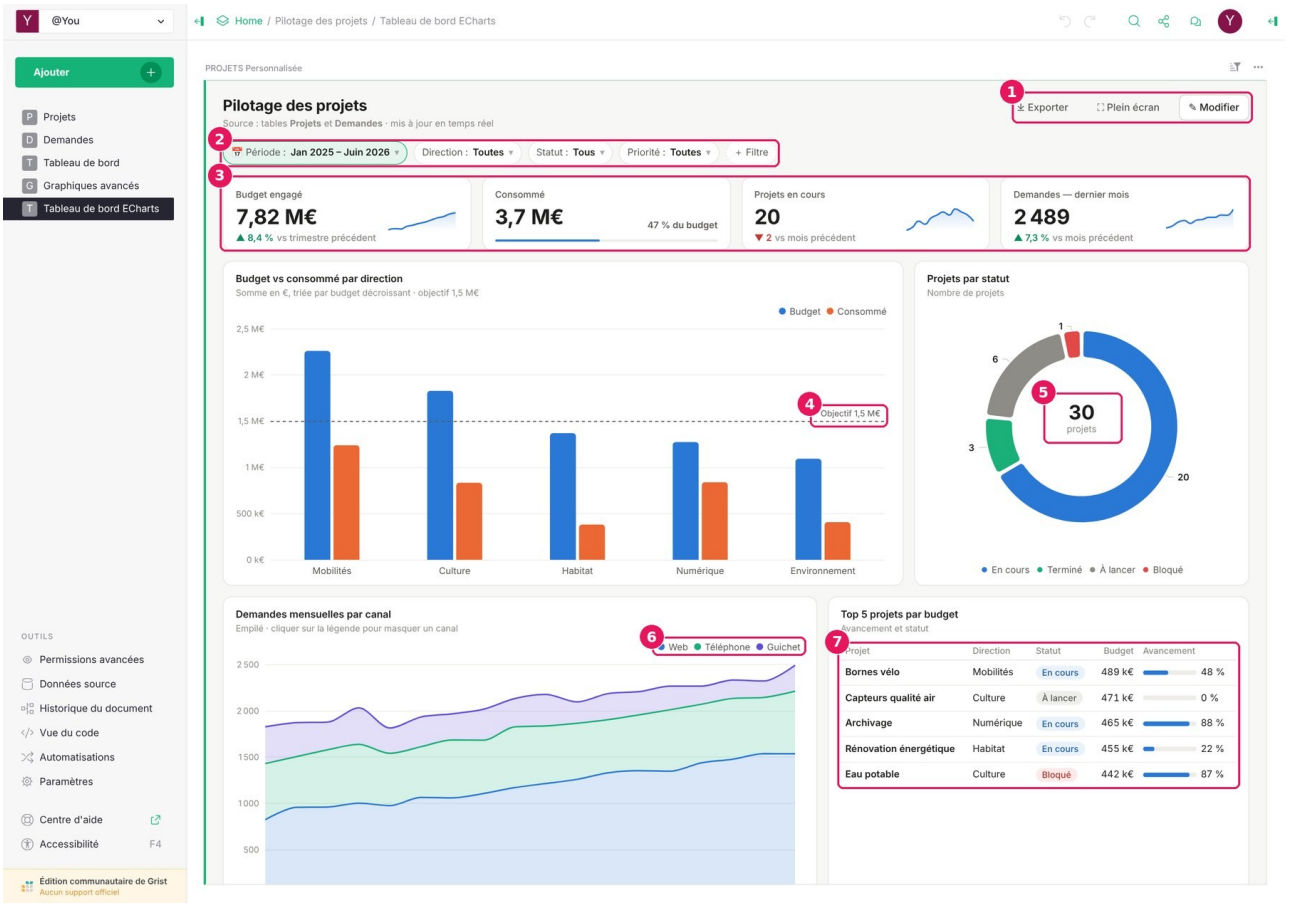


Figure 15 — *Cible — le widget dans Grist, alimenté par les tables « Projets » et « Demandes » : (1) actions Exporter / Plein écran / Modifier, (2) filtres globaux, (3) rangée de KPI avec tendance et comparaison, (4) ligne d'objectif, (5) total au centre du donut, (6) légende interactive, (7) tableau « Top 5 » avec statuts colorés et barres de progression.*

5.2 Modes lecture et édition

Le widget s'ouvre en **mode lecture**. Les utilisateurs ayant le droit d'éditer le document voient le bouton « Modifier », qui bascule en **mode édition** ; les autres ne le voient pas. Les modifications ne sont enregistrées qu'au clic sur « Enregistrer » (annulation possible).

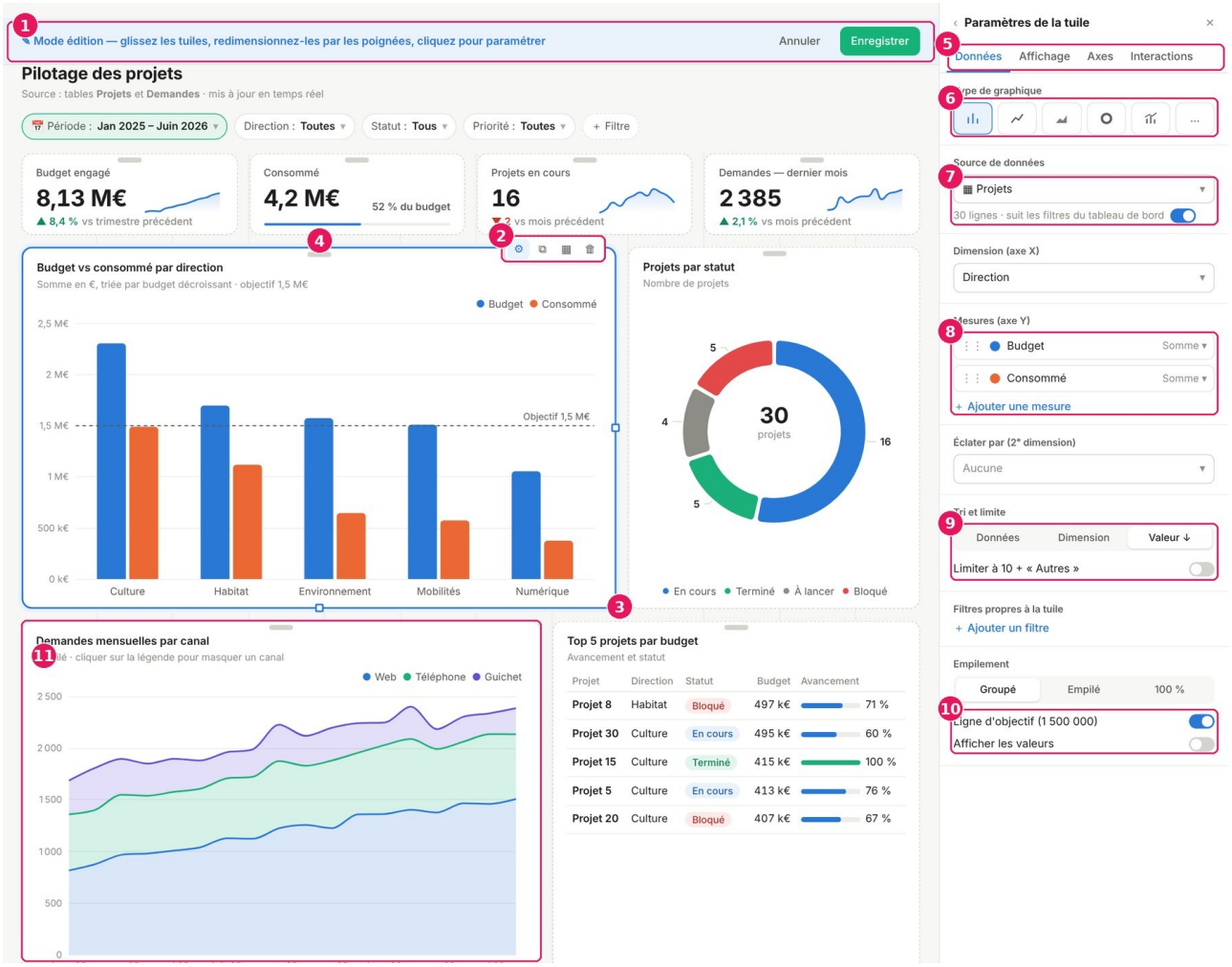


Figure 16 — *Mode édition (maquette) — voir la légende ci-dessous.*

#	Observation
1	Bandeau d’édition : rappel des gestes possibles, boutons Annuler / Enregistrer.
2	Barre d’outils de la tuile sélectionnée : paramètres, dupliquer, voir les données, supprimer.
3	Poignées de redimensionnement (droite, bas, coin) avec aimantation à la grille de 12 colonnes.
4	Poignée de déplacement (glisser-déposer) ; les autres tuiles se réorganisent automatiquement.
5	Panneau de paramètres à quatre onglets : Données, Affichage, Axes, Interactions.
6	Changement du type de graphique sans perte de la configuration des données.
7	Source de données (table Grist) et option « suit les filtres du tableau de bord ».
8	Mesures avec agrégation par mesure (somme, moyenne, nombre…), couleur et ordre modifiables.
9	Tri (ordre des données, dimension, valeur) et limitation au Top N avec regroupement « Autres ».
10	Options d’affichage fréquentes en accès direct : ligne d’objectif, étiquettes de valeur.
11	Grille de fond pointillée visible uniquement en édition ; tuiles en bordure tiretée.

5.3 Catalogue des tuiles

Le sélecteur de tuiles regroupe les visualisations **par intention** (indicateurs, comparer, évolution, répartition, relations, tableaux) et met en avant les types suggérés à partir des colonnes choisies, sur le modèle de la recommandation de Metabase.

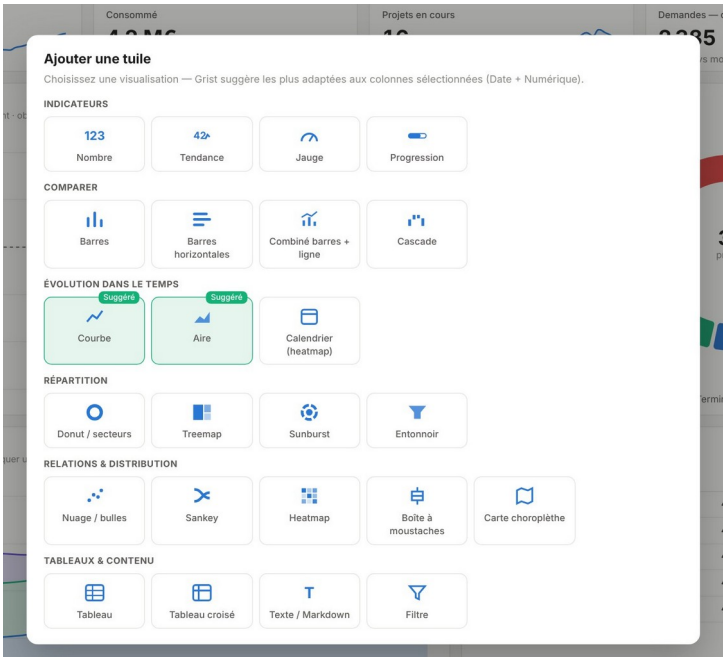


Figure 17 — *Sélecteur « Ajouter une tuile » (maquette) : regroupement par intention et suggestions selon les types de colonnes.*

Tableau 4 — *Catalogue des tuiles et lotissement*

Famille	Type de tuile	Données requises	Paramètres spécifiques	Lot
Indicateurs	Nombre (KPI)	1 mesure	Préfixe/suffixe, unité, abréviation k/M, couleur conditionnelle	MVP
	Tendance	1 date + 1 mesure	Comparaison période précédente ou valeur fixe, sens favorable, sparkline	MVP
	Jauge	1 mesure	Min/max, segments colorés et libellés	V1
	Progression	1 mesure + objectif	Objectif fixe ou colonne, couleur si dépassé	V1
Comparer	Barres / barres horizontales	1 dimension + n mesures	Groupé / empilé / 100 %, objectif, valeurs, Top N	MVP
	Combiné barres + ligne	1 dimension + 2 mesures	Type par série, second axe (désactivé par défaut)	V1
	Cascade	1 dimension + 1 mesure	Couleurs hausse/baisse/total	V2
Évolution	Courbe, aire	1 date + n mesures	Granularité, lissage, points, cumul, tendance, zoom	MVP
	Calendrier (heatmap)	1 date + 1 mesure	Année, rampe séquentielle	V2
Répartition	Donut / secteurs	1 dimension + 1 mesure	Total central, % ou valeur, regroupement des petites parts	MVP
	Treemap, sunburst	1-2 dimensions + 1 mesure	Niveaux, zoom, libellés	V1
	Entonnoir	1 dimension (étapes) + 1 mesure	Ordre des étapes, taux de conversion	V2
Relations	Nuage / bulles	2 mesures (+ taille, couleur)	Taille des bulles, régression	V1
	Sankey	Source + cible + mesure	Alignement, couleur des liens	V2
	Heatmap, boîte à moustaches	2 dimensions + mesure / 1 mesure	Rampe, quartiles	V2

Famille	Type de tuile	Données requises	Paramètres spécifiques	Lot
	Carte choroplèthe	Code géographique + mesure	Fond GeoJSON configurable	V2
Tableaux & contenu	Tableau, tableau croisé	Colonnes / lignes × colonnes	Mise en forme conditionnelle, barres dans les cellules, totaux	MVP / V1
	Texte / Markdown, titre de section, filtre	—	Markdown, variables (date du jour, nb de lignes)	MVP

5.4 Paramétrage d'une tuile

Chaque paramètre exposé dans l'interface correspond à une option ECharts documentée, ce qui garantit la faisabilité et facilite la maintenance. Le tableau ci-dessous fixe le contenu de chaque onglet.

Tableau 5 — *Paramètres d'une tuile et correspondance ECharts*

Onglet	Paramètre (UI)	Valeurs	Correspondance ECharts / traitement
Données	Source	Table Grist, option « suit les filtres globaux »	`docApi.fetchTable()` + abonnement aux changements
	Dimension (axe X)	Colonne ; pour une date : jour, semaine, mois, trimestre, année	Regroupement côté widget ; `xAxis.type: time | category`
	Mesures	Colonne + agrégation : nombre, nombre distinct, somme, moyenne, médiane, min, max, cumul	Pipeline d'agrégation (frame.ts) → `series[].data`
	Éclater par	2e dimension (une série par valeur)	Pivot → n séries, couleurs stables par valeur
	Tri et limite	Données, dimension, valeur ↑↓ ; Top N + « Autres »	Tri du jeu de données avant rendu
	Filtres de tuile	Conditions par colonne (=, ≠, contient, entre, vide…)	Filtrage avant agrégation
Affichage	Empilement	Groupé, empilé, 100 %	`series[].stack`, normalisation
	Ligne d'objectif	Valeur fixe ou colonne, libellé	`series[].markLine`
	Étiquettes de valeur	Aucune, toutes, extrêmes ; format auto/compact/complet	`series[].label` + formateur FR
	Couleurs	Palette du thème, couleur par série, couleur conditionnelle	`color[]`, `itemStyle.color`, `visualMap`
	Légende, infobulle	Position, colonnes additionnelles dans l'infobulle	`legend`, `tooltip.formatter`
	Courbes	Lissage, points, aire, tendance linéaire	`smooth`, `showSymbol`, `areaStyle`, régression
Axes	Titres et libellés	Afficher, texte, rotation auto	`xAxis.name`, `axisLabel`
	Échelle	Linéaire, logarithmique, min/max, détacher de zéro	`yAxis.type`, `min`, `max`, `scale`
	Format des nombres	Hérité de la colonne Grist (devise, %, décimales) ou forcé	Formateur basé sur `Intl.NumberFormat("fr-FR")`
Interactions	Clic sur un élément	Aucun, filtrer le tableau de bord, ouvrir les enregistrements	`chart.on("click")` → filtre croisé ou `setCursorPos` / `setSelectedRows`

Onglet	Paramètre (UI)	Valeurs	Correspondance ECharts / traitement
	Zoom	Barre de défilement temporelle, zoom molette	`dataZoom` (slider / inside)
	Export	PNG, SVG, CSV des données agrégées	`getDataURL()`, `renderToSVGString()`, CSV

5.5 Données et règles de calcul

- **Lecture** : le widget demande l'accès « document complet » pour lire plusieurs tables ; il lit les données **avec les droits de l'utilisateur courant** (les règles d'accès avancées de Grist s'appliquent : un lecteur ne voit que ses lignes).
- **Temps réel** : toute modification des tables sources met à jour les tuiles concernées en moins d'une seconde, sans rechargement.
- **Types Grist** : Référence et Liste de références affichent la colonne d'affichage ; Liste de choix compte une occurrence par choix ; les valeurs vides forment une catégorie « (vide) » masquable ; les dates suivent le fuseau du document.
- **Formats** : le format défini dans la colonne Grist (devise, pourcentage, décimales, format de date) est repris par défaut dans les axes, étiquettes et infobulles.
- **Couleurs des Choix** : quand la dimension est une colonne Choix, les couleurs définies dans Grist sont proposées en priorité, pour une cohérence entre tableau et graphique.
- **Volumétrie** : agrégation côté navigateur jusqu'à 100 000 lignes ; au-delà, recommandation d'utiliser une table récapitulative Grist comme source.
- NOUVEAU **Agrégation côté serveur** quand l'utilisateur a un accès complet au document, via l'endpoint SQL de Grist (voir § 7.2) ; agrégation dans le navigateur sinon, pour respecter les règles d'accès.

5.6 Filtres globaux et interactions

- **Bandeau de filtres** en haut du tableau de bord : période (préréglages « 30 derniers jours », « Ce trimestre », « Cette année », plage libre), listes de valeurs (Choix, Référence), recherche texte. Chaque filtre cible une colonne par table source ; une tuile peut s'en affranchir.
- **Filtre croisé** : un clic sur une barre ou une part de donut filtre les autres tuiles ; un rappel « Filtré par : Direction = Habitat ✕ » s'affiche dans le bandeau.
- **Retour aux données** : action « Voir les enregistrements » qui sélectionne les lignes concernées dans une section liée (ou ouvre une fiche), rendant le tableau de bord actionnable.
- **Liaison Grist** : le widget peut aussi être la cible d'un « Sélectionner par » (par exemple, filtrer tout le tableau de bord à partir d'une liste de directions dans une autre section).

5.7 Mise en page

- Grille de **12 colonnes**, hauteur par pas de 96 px ; tailles par défaut : KPI 3 × 1, graphique 6 × 4, tableau 6 × 4.
- **Sections** (titre + texte d'analyse) et **onglets** pour organiser les tableaux de bord longs, comme dans Metabase.
- **Responsive** : 12 colonnes au-delà de 1 200 px, 6 colonnes entre 768 et 1 199 px, empilement vertical en dessous (consultation mobile).
- **Plein écran / mode présentation** et rafraîchissement automatique optionnel pour un affichage sur écran mural.

5.8 Export et partage

- Par tuile : PNG (haute définition), SVG (vectoriel) et CSV des données agrégées.

- Tableau de bord complet : impression / PDF A4 paysage avec en-tête (titre, filtres actifs, date d'extraction).
- Le partage suit celui du document Grist (lien, rôles) ; aucun partage public propre au widget n'est introduit.

6. Design et charte graphique

6.1 Principes

1. **Les données d'abord** : encre minimale, grilles très claires, pas de cadre d'axe, pas d'effet 3D ni d'ombre portée sur les graphiques.
2. **Natif Grist** : typographie et surfaces cohérentes avec l'interface de Grist ; le vert Grist est réservé aux actions (boutons, filtres actifs), jamais aux séries de données.
3. **Hiérarchie constante** : titre → filtres → KPI → graphiques → détail.
4. **Lisible par tous** : palette validée pour les daltonismes, contrastes conformes, alternative tabulaire pour chaque graphique.

6.2 Palette

La palette catégorielle comporte **8 teintes attribuées dans un ordre fixe** (jamais recyclées ; au-delà, regroupement « Autres »). Elle a été vérifiée avec un outil de contrôle des distances perceptuelles : séparation minimale entre teintes voisines ΔE 9,1 pour les daltonismes (cible ≥ 8) et 19,6 en vision normale (seuil ≥ 15). Trois teintes ayant un contraste inférieur à 3:1 sur fond clair, les étiquettes directes ou la vue tableau restent disponibles. Une couleur suit **l'entité, pas son rang** : filtrer ne repeint pas les séries restantes.



Figure 18 — *Palette proposée : catégorielle (clair et sombre), séquentielle, divergente et statuts. Les couleurs de statut sont réservées et toujours accompagnées d'une icône ou d'un libellé.*

6.3 Jetons de design et anatomie d'une tuile

Tableau 6 — *Jetons de design*

Élément	Spécification
Typographie	Inter (ou police système de Grist) ; titre de tuile 13 px semi-gras ; description 11,5 px gris ; KPI 27 px gras, chiffres tabulaires
Tuile	Fond blanc, bordure 1 px #E6E5E1, rayon 12 px, marge interne 14 × 16 px, espacement de grille 14 px
Barres	Coins supérieurs arrondis 4 px, largeur max 34 px, écart de 2 px entre barres adjacentes ou segments empilés

Élément	Spécification
Lignes et aires	Trait 2 px, points masqués sauf au survol (≥ 8 px), aire à 18 % d'opacité, lissage léger
Donut	Anneau 50–70 %, séparation 1,5°, coins arrondis, total au centre, étiquettes en valeur
Axes et grille	Pas de ligne d'axe ni de graduations ; grille horizontale #EEEDE9 ; libellés 11 px gris
Légende	Pastilles rondes 9 px, en haut à droite ; cliquable pour masquer une série ; absente pour une série unique
Infobulle	Fond blanc, rayon 8 px, ombre douce ; valeurs formatées en français ; ligne de repère verticale sur les séries temporelles
Nombres	Espace insécable pour les milliers, virgule décimale, abréviations « k€ », « M€ » ; pourcentages « 52 % »
Double axe	Désactivé par défaut (source fréquente de mauvaise lecture) ; autorisé uniquement pour le graphique combiné, avec avertissement

6.4 Mode sombre

Le widget suit le thème de Grist (clair / sombre) transmis par l'API des widgets. Les couleurs du mode sombre ne sont **pas une inversion automatique** : ce sont les mêmes teintes, recalées pour la surface sombre et vérifiées séparément.

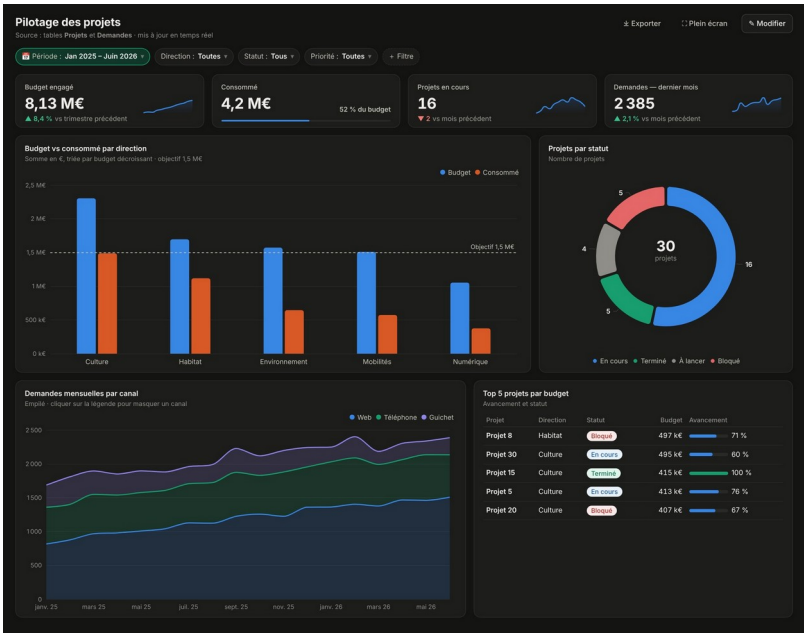


Figure 19 — *Le même tableau de bord en mode sombre (maquette).*

6.5 Accessibilité

- Conformité visée **RGAA 4.1 / WCAG 2.1 AA** pour l'interface du widget (navigation au clavier, focus visible, libellés des contrôles).
- Graphiques : description ARIA générée par ECharts (`aria.enabled`), motifs de remplissage optionnels (`aria.decal`) pour l'impression et les daltonismes, et **vue tableau** accessible pour chaque tuile.
- L'identité d'une série n'est jamais portée par la seule couleur : légende pour 2 séries ou plus, étiquettes directes jusqu'à 4 séries.

7. Exigences non fonctionnelles

Tableau 7 — *Exigences non fonctionnelles*

Domaine	Exigence	Critère de mesure
Performance	Premier affichage d'un tableau de bord de 8 tuiles sur 10 000 lignes	< 1,5 s sur un poste standard
	Mise à jour après modification d'une cellule	< 1 s
Compatibilité	Navigateurs	Dernières versions de Chrome, Edge, Firefox, Safari
	Grist	grist-core ≥ 1.7 auto-hébergé et Grist SaaS ; installation par la liste de widgets (`GRIST_WIDGET_LIST_URL`) ou par URL
Sécurité	Exécution	Widget isolé dans l'iframe Grist ; aucune requête vers un service tiers ; aucune librairie chargée depuis un CDN en production
	Droits	Lecture avec les droits de l'utilisateur ; écriture limitée aux options du widget, réservée aux éditeurs
Localisation	Langues	Français et anglais, selon la langue de l'interface Grist ; formats selon la locale du document
Licence	Distribution	Code sous Apache 2.0, publiable dans le dépôt de widgets Grist
Qualité	Tests	Tests unitaires du pipeline d'agrégation (> 80 %), tests de rendu visuel (captures) par type de graphique, tests de bout en bout dans grist-core
Maintenabilité	Code	TypeScript strict, un module par famille de graphique, schéma de configuration versionné et migrable

7.1 Architecture technique

- **Interface** : TypeScript, React 18 et Mantine (comme le prototype Rath), compilés avec Vite en un bundle statique autonome.
- **Rendu** : ECharts 6 en import modulaire (seuls les graphiques utilisés sont chargés), rendu SVG par défaut.
- **Grille** : `react-grid-layout` (licence MIT) pour le glisser-déposer et le redimensionnement.
- **Données** : `grist-plugin-api` (`fetchTable`, `onRecords`, `onOptions`, `setOptions`, `setCursorPos`, `setSelectedRows`).
- **Modules** : couche données (lecture, typage, cache) → moteur de requête (filtres, regroupements, agrégations) → registre des visualisations → constructeurs d'options ECharts → interface de réglages.

Exemple de configuration enregistrée dans les options du widget :

```
{
  "version": 1,
  "title": "Pilotage des projets",
  "filters": [{ "id": "periode", "type": "dateRange", "targets": { "Demandes": "Mois", "Projets":
"Debut" } }],
  "tiles": [{
    "id": "t2", "type": "bar", "layout": { "x": 0, "y": 1, "w": 8, "h": 4 },
    "title": "Budget vs consommé par direction",
    "data": { "table": "Projets", "dimension": "Direction",
              "measures": [{ "col": "Budget", "agg": "sum" }, { "col": "Consomme", "agg": "sum" }
],
              "sort": { "by": "value", "dir": "desc" }, "followGlobalFilters": true },
    "display": { "stack": "none", "goal": { "value": 1500000, "label": "Objectif 1,5 M€" } },
    "interactions": { "click": "crossFilter" }
  }]
}
```

7.2 Compléments techniques vérifiés dans le code de grist-core NOUVEAU v1.1

Les points suivants ont été vérifiés directement dans le code source de grist-core (version 1.7.21) et modifient ou précisent l'architecture décrite en § 5.5 et § 7.1.

Tableau 8 — *Mécanismes de l'API Grist utiles au widget*

Mécanisme	Constat dans grist-core	Usage proposé pour le widget
Endpoint SQL `POST /api/docs/{id}/sql`	Requêtes SELECT uniquement ; délai maximal de 1 s par défaut (`GRIST_SQL_TIMEOUT_MSEC`) ; exige le droit de copier tout le document (`canCopyEverything`), donc refusé si des règles d'accès masquent des données à l'utilisateur.	Stratégie hybride : si l'utilisateur a un accès complet, agréger côté serveur (GROUP BY) ; sinon, agréger dans le navigateur à partir de `fetchTable`, qui applique les règles d'accès. Repousse la limite de 100 000 lignes pour la plupart des usages.
Jeton d'accès `docApi.getAccessToken()`	Jeton court, utilisable en lecture seule (`readOnly: true`) pour appeler l'API REST du document.	Autorise l'appel SQL ci-dessus depuis l'iframe, sans clé d'API stockée.
Thème `onThemeChange`	Le thème Grist (clair/sombre, couleurs) est transmis au widget et appliqué sous forme de variables CSS.	Le widget bascule automatiquement en mode sombre et reprend les couleurs d'interface de Grist.
Configuration `ready({ onEditOptions })`	Grist affiche alors un bouton « Ouvrir la configuration » dans son panneau de droite.	Ce bouton ouvre le mode édition du tableau de bord, en plus du bouton « Modifier » interne.
Catalogue de widgets `GRIST_WIDGET_LIST_URL`	Une instance liste ses widgets dans un manifeste JSON (nom, URL, niveau d'accès, `renderAfterReady`).	Une instance auto-hébergée (collectivité, administration) peut proposer le widget à tous ses utilisateurs sans saisie d'URL.

Écosystème existant. Le catalogue officiel de widgets Grist contient « Advanced charts » (éditeur Plotly, mis à jour en juin 2025), « Pivot table » et, depuis février 2026, « Vibe View » (gabarits de pages et de catalogues). **Aucun widget de tableau de bord ni aucun widget fondé sur ECharts n'a été trouvé** : la proposition ne fait pas doublon.

Attentes de la communauté. Le forum Grist contient des demandes récurrentes qui sont toutes couvertes par ce cahier des charges : créer un graphique depuis une table récapitulative, graphiques à plusieurs axes, export des graphiques en PNG, tableau croisé dynamique.

8. Lotissement, planning et risques

Tableau 9 — *Lotissement indicatif (charges en jours-personne, développeur front-end confirmé)*

Lot	Contenu	Charge estimée	Livrable
0. Cadrage	Validation du cahier des charges, maquettes détaillées, schéma de configuration	5 j	Maquettes validées
1. MVP	Grille 12 colonnes, modes lecture/édition, KPI + tendance, barres, lignes, aires, donut, tableau, texte ; panneau Données/Affichage ; filtre période	35 j	Widget installable, document de démonstration
2. V1	Filtres globaux complets, filtre croisé, retour aux enregistrements, jauge, progression, combiné, treemap, nuage, tableau croisé ; onglet Axes ; export PNG/SVG/CSV ; mode sombre	30 j	Version publiable dans la galerie de widgets
3. V2	Cascade, entonnoir, Sankey, heatmap, calendrier, boîte à moustaches, carte choroplèthe ; sections et onglets ; impression PDF ; mode présentation	25 j	Version complète
Transverse	Accessibilité, tests, documentation utilisateur FR/EN, recette	15 j	Rapport de recette, guide

Total indicatif : **≈ 110 jours-personne**, soit environ 5 à 6 mois pour un développeur, ou 3 mois pour un binôme développeur + designer. La réutilisation du prototype Rath réduit le risque et la charge des lots 1 et 2.

Tableau 10 — *Principaux risques*

Risque	Impact	Mesure de réduction
Performance sur de gros volumes (agrégation dans le navigateur)	Moyen	Échantillonnage ECharts, mise en cache, recommandation des tables récapitulatives au-delà de 100 000 lignes
Accès « document complet » jugé trop large par certains administrateurs	Moyen	Mode dégradé à une seule table avec l'accès « lecture de la table sélectionnée »
Évolution de l'API des widgets Grist	Faible	Couche d'accès aux données isolée, tests de bout en bout sur chaque version de grist-core
Dérive du périmètre vers un outil de BI	Élevé	Catalogue figé par lot, arbitrage en comité ; pas de SQL libre
Taille du bundle	Faible	Import modulaire d'ECharts, chargement différé des types avancés

8.1 Critères d'acceptation

1. Un concepteur crée, sans documentation, un tableau de bord de 4 KPI et 3 graphiques en **moins de 10 minutes** (test utilisateur sur 5 personnes).
2. Les valeurs affichées sont **identiques** à celles calculées par une table récapitulative Grist sur le même jeu de données (tests automatisés).
3. Une modification de cellule est répercutée dans les tuiles concernées en moins d'une seconde.
4. Un utilisateur soumis à une règle d'accès ne voit dans le tableau de bord **que les données auxquelles il a droit**.
5. Le tableau de bord se copie avec le document, survit à un export/import `.grist` et à une restauration d'instantané.
6. Audit d'accessibilité sans non-conformité bloquante ; contrôle de la palette en clair et en sombre.

Annexe — Sources

- Notion, centre d’aide : « Chart view » (notion.com/help/charts) et « Dashboards view » (notion.com/help/dashboards).
- Airtable, support : « Airtable interface layout: Dashboard » ; page produit Interface Designer (airtable.com/platform/interface-designer).
- NocoDB, documentation : « Dashboards » et « Widgets » ; journal des versions 2025.07.0.
- Metabase, documentation : « Visualizing results », « Line, bar and area charts », « Dashboards introduction » ; dépôt github.com/metabase/metabase (dépendance echarts 6.1.0).
- Apache ECharts, documentation et exemples (echarts.apache.org).
- grist-core (code source, `app/client/components/ChartView.ts`, dépendance plotly.js-basic-dist 2.13.2) ; manifeste des widgets Grist (gristlabs/grist-widget).
- Prototype Rath : dépôt Rath et déploiement rath-rath-client-8vrf.vercel.app.
- NOUVEAU Baserow : « Dashboards overview » (baserow.io/user-docs) et notes de version 1.32 et 2.4 ; SeaTable : aide « Statistiques » (seatable.com/help/statistics).
- NOUVEAU Airtable : « AI-generated interface elements » et « Using Omni AI » (support.airtable.com) ; forum communautaire Grist (community.getgrist.com).
- NOUVEAU grist-core : `app/server/lib/runSQLQuery.ts`, `app/server/lib/DocApi.ts`, `app/plugin/grist-plugin-api.ts`.

Les captures des solutions concurrentes sont reproduites à des fins d’analyse comparative ; elles restent la propriété de leurs éditeurs. Les captures Grist, Rath et les maquettes ont été réalisées pour ce document le 20 octobre 2026.